A Deterministic Validation Loop for LLM-Generated Scientific Code: Framework, Implementation, and Empirical Validation

Rice, J.

Corresponding codebase:
Framework: https://github.com/kepiCHelaSHen/context-hacking (MIT License)
Validation: https://github.com/kepiCHelaSHen/SIMSIV
DOI: 10.5281/zenodo.19065475

Companion video: https://www.youtube.com/watch?v=hkgjsoVkUdg

March 2026

Abstract

Large Language Models generate scientific code from training priors rather than user-defined specifications, silently substituting plausible but incorrect numerical coefficients—a failure mode we term *specification drift*. We present a five-component deterministic validation framework that detects and eliminates drift while preserving generative capacity: (1) Immutable Specification, (2) Prior-Drift Detection, (3) Adversarial Role Separation, (4) Statistical Validation, and (5) Persistent State with Termination Control. In a controlled experiment across two frontier LLMs (GPT-4o, Grok-3; $n = 10$ trials per model per coefficient), 95 of 96 blind measurements drifted from specification (Fisher's exact $p = 4.0\times10^{-10}$). Under the full framework, zero committed drift occurred across 7 checked parameters, validated over 120 simulation runs ($\sigma = 0.030$, CV = 6.0%). The framework's statistical gating component independently caught a false positive before it could be published. We validate on SIMSIV (https://github.com/kepiCHelaSHen/SIMSIV), a 7,663-line agent-based simulation of human social evolution currently under review at JASSS. These results demonstrate that specification drift is a context problem amenable to deterministic engineering controls, with implications for any domain where LLMs generate calibrated scientific software.

1 Introduction

1.1 The Problem

When Large Language Models generate scientific software, they produce code that compiles, passes unit tests, and appears structurally sound—yet silently substitutes incorrect numerical coefficients for those defined in the target specification. A simulation model calibrated with an empathy modulation coefficient of 0.15, grounded in published literature [de Waal, 2008], receives the value 0.20 or 0.30 from an LLM. The code runs. The dynamics change. No error is raised.

This failure mode—*specification drift*—is not hypothetical. Across 96 controlled measurements in this study, frontier LLMs produced the correct coefficient exactly once.

The implications extend beyond simulation science. Clinical trial software with drifted dosage thresholds, financial models with substituted risk coefficients, and engineering simulations with incorrect material constants all represent scenarios where LLM-generated code could silently

invalidate results. In each case, the failure is undetectable by standard software quality assurance: the code compiles, the types check, the tests pass. Only direct comparison against the authoritative specification reveals the error.

1.2 The Core Insight: Weakness as Strength

The conventional response to LLM unreliability is suppression: constrain outputs through reduced temperature, more detailed prompting, and post-hoc validation. This treats the model's generative prior as purely adversarial.

We propose an inversion. The same property that causes drift—generation from training priors rather than from specifications—is also the property that makes LLMs creative. A model that generates only from its specification is a copy-paste engine. A model that generates from its prior produces novel code structures, variable names, architectural patterns, and abstraction choices. The prior is not a defect; it is a generative resource.

The analogy is to genetic mutation in evolutionary biology. Mutation is individually deleterious (most mutations are harmful), yet it is the substrate upon which natural selection operates. Without mutation, there is no adaptation. Without the LLM's prior-driven generation, there are no novel proposals to evaluate.

Our framework does not suppress the prior. It *governs* it. A Builder role generates freely from its prior, producing complete implementations with creative latitude. A Critic role validates every coefficient against a frozen specification. The creative generation and the specification compliance occur in the same context window, in the same session, from the same model. The governance mechanism is a *context hack*: the specification is injected into the model's active attention through an adversarial role constraint, redirecting attention from "what seems reasonable" to "what the specification says."

Our experimental data validates this framing. Under the framework, the Builder proposed an empathy coefficient of 0.20—the same drifted value produced by unstructured models. The Critic caught it before commit. The Builder's creative structural contributions (code organisation, naming, abstraction) survived; only the specification-violating coefficients were corrected. The weakness was governed, not suppressed.

1.3 Chronology: The Framework Preceded the Measurement

A critical clarification: we did not observe specification drift and then design a countermeasure. The framework was the *operating procedure* under which SIMSIV was constructed. The v1 codebase was built with these principles in mind; the v2 codebase (7,663 lines) was constructed by a fully autonomous LLM loop running the framework's Builder/Critic/Reviewer protocol while the human operator was asleep. The controlled drift experiment (Section 4) was conducted *afterward* to quantify what the framework had been preventing.

This chronology matters. The false positive caught in Section 4.8 was not detected by retrospective analysis—it was caught in real time by the framework's sigma-gating component during autonomous operation. The dead-end register, the forced replication, and the eventual discovery of the real finding all occurred without human oversight. The framework did not need a human to function. It needed a human to write the frozen specification. After that, the validation loop operated autonomously and correctly.

1.4 Paper Structure

Section 2 reviews related work. Section 3 presents the five-component framework with formal definitions and implementation mappings. Section 4 reports the empirical validation: a three-condition controlled experiment measuring specification drift across frontier LLMs, a 120-run multi-seed validation, and a documented false positive caught by the framework's statis-

tical gating. Section 5 discusses implications, limitations, and future directions. We validate the framework on SIMSIV (https://github.com/kepiCHelaSHen/SIMSIV), a 7,663-line agent-based simulation of human social evolution, under review at JASSS (submission 2026:81:1).

2 Related Work

2.1 Prompt Engineering and Output Structuring

Techniques such as chain-of-thought prompting [Wei et al., 2022], few-shot exemplars, and structured output formatting improve LLM reliability on reasoning tasks. However, they operate within a single inference step and provide no mechanism for detecting when a model generates from its training prior rather than from user-provided specifications. A model instructed to "implement this coefficient as 0.15" may still produce 0.30 if its prior is sufficiently strong—an effect we quantify in Section 4.3.

2.2 Retrieval-Augmented Generation

RAG [Lewis et al., 2020] injects relevant documents into the model's context before generation, partially addressing the context deficit that enables drift. Our Condition B results (Section 4.5) demonstrate that making the correct specification visible does eliminate simple coefficient drift. However, RAG provides no adversarial review, no multi-seed statistical validation, and no mechanism for detecting structural errors (formula-level mistakes, scope creep) that persist even when correct coefficients are visible.

2.3 Self-Reflection and Agent Frameworks

Reflexion [Shinn et al., 2023] enables models to reflect on failed attempts and adjust strategies. LATS [Zhou et al., 2023] combines language agents with tree search for exploration. DSPy [Khattab et al., 2023] optimises prompt pipelines programmatically. Constitutional AI [Bai et al., 2022] uses principle-based self-revision. These approaches advance LLM reliability through self-correction, search, or principled constraints.

2.4 What Is Different Here

None of these approaches treat prior-generation as a *detectable signal* to be exploited. They seek to improve LLM outputs through better prompting, richer context, or iterative refinement. Our framework treats the gap between what the model generates (its prior) and what the specification requires (the frozen ground truth) as the primary diagnostic instrument. The prior is not noise to be filtered; it is a measurement to be compared against a reference. This inversion—from "improve the model" to "measure the model against a specification and use the divergence as the error signal"—is the distinguishing contribution.

3 The Five-Component Framework

We define a deterministic validation loop comprising five components. The framework enforces specification compliance through: (1) an immutable specification that defines the ground truth and cannot be modified by any agent; (2) prior-drift detection that measures divergence between generated and specified values; (3) adversarial role separation that prevents self-confirmation by partitioning generation and validation; (4) multi-seed statistical validation that gates acceptance on reproducibility, not single-run results; and (5) persistent state

with termination control that prevents repeated errors and ensures the loop halts. Each component addresses a specific failure mode of LLM-generated scientific code. We provide formal definitions, implementation mappings to the Context Hacking Protocol (CHP; https://github.com/kepiCHelaSHen/context-hacking), and references to SIMSIV as the instantiation.

3.1 Component 1: Immutable Specification (Ground Truth Lock)

Objective. Prevent silent drift away from intended scientific definitions.

Formal definition. Let $S = \{s_1, s_2, \ldots, s_n\}$ be the set of specified parameters, where each $s_i = (k_i, v_i, \ell_i, c_i)$ consists of a parameter name k_i, a frozen value v_i, a source-code location ℓ_i (file and line number), and a literature citation c_i. The specification S is declared immutable: no agent in the generation loop may modify any v_i.

$$\forall i, \forall t: \quad v_i^{(t)} = v_i^{(0)} \tag{1}$$

where t indexes generation turns.

Rationale. Without an immutable reference, there is nothing to drift *from*. The specification must be external to the LLM's generation process and unmodifiable by it.

CHP implementation. Layer 3 (Frozen Code Forcing). Published or validated code is placed in a `frozen/` directory that no agent may edit. All new code must compose with frozen files. Every coefficient in generated code must trace to a specific (ℓ_i, c_i) pair.

SIMSIV instantiation. The frozen specification is JASSS submission 2026:81:1 and the `frozen/spec` files in the SIMSIV repository. Five coefficients across four source files, each traced to a literature citation (Table 1).

Table 1: Ground truth specification: five frozen coefficients.

#	Parameter	v_i	ℓ_i	c_i
1	Empathy modulation	0.15	`resources.py:289`	de Waal [2008]
2	Cooperation norm mod.	0.10	`resources.py:292`	Boyd & Richerson [1985]
3	Social skill trade bonus	0.10	`clan_trade.py:330`	Wiessner [1982]
4	Cohesion defence bonus	0.20	`clan_raiding.py:610`	Bowles [2006]
5	No. prosocial traits	4	`clan_selection.py:82-87`	Price [1970]

3.2 Component 2: Prior-Drift Detection

Objective. Detect when the LLM generates from training priors instead of from the specification.

Formal definition. For each generated parameter value $\hat{v}_i$, define:

$$\text{Drift}_i = \mathbb{1}(\hat{v}_i \neq v_i) \tag{2}$$

$$\text{Drift Magnitude}_i = |\hat{v}_i - v_i| \tag{3}$$

The aggregate drift rate over N measurements is:

$$D = \frac{1}{N} \sum_{i=1}^{N} \text{Drift}_i \tag{4}$$

A drift rate $D > 0$ triggers a hard block: the generated code may not be committed until all $\text{Drift}_i = 0$.

Rationale. The model's prior is strong enough to produce deterministic wrong answers (Section 4.4: Grok-3 produced $\hat{v} = 0.30$ on 10/10 trials for a 0.10 parameter). This determinism makes drift *more* detectable, not less. If you know the model's prior, you know what a drifted output looks like.

CHP implementation. Layer 1 (Prior-as-Detector). The divergence between the LLM's generated value and the frozen specification is treated as the primary error signal. When output matches a known training prior rather than the frozen spec, the Critic flags the specific divergence with reference to the source-code line.

SIMSIV instantiation. The Critic role is provided the frozen spec and instructed to compare every coefficient against specific source-code lines. Section 4.3 documents the drift rates; Section 4.6 documents the Critic catching every drifted proposal.

3.3 Component 3: Adversarial Role Separation (Builder / Critic)

Objective. Eliminate self-confirmation and enforce internal challenge.

Formal definition. The generation process is partitioned into three roles with distinct authorities (Table 2).

Table 2: Adversarial role structure.

Role	Function	Authority
Builder	Generates candidate implementation	Proposer only
Critic	Validates against S; structural consistency	Hard-blocker
Reviewer	Audits code quality and architecture	Advisory

A build is accepted if and only if:

$$\text{Approved} = \left(\bigwedge_{i=1}^{n} \text{Drift}_i = 0 \right) \wedge \text{CriticPass} \wedge \text{ReviewerPass} \tag{5}$$

Rationale. A single model asked to both generate and validate will confirm its own output (sycophancy bias). Separating generation from validation—even within the same model instance using role prompts—breaks the self-confirmation loop. The Builder *should* generate from its prior (this is its creative contribution). The Critic's job is to catch the specification violations in what the Builder produces.

CHP implementation. Layer 2 (Synthetic Dialectic). The Critic prompt reads: "Assume the build failed until proven otherwise. Argue AGAINST the science before scoring it." The Critic cross-references every coefficient against the frozen source file and line number.

SIMSIV instantiation. Table 7 (Section 4.6) documents six friction events where the Builder proposed drifted values and the Critic hard-blocked each one. The Builder's proposed empathy coefficient (0.20) falls within Grok-3's Condition A drift range (0.20–0.25), confirming that the Builder exhibits the same drift as unstructured models—but the Critic catches it before commit.

3.4 Component 4: Statistical Validation (Multi-Seed + Variance Gating)

Objective. Prevent false positives and ensure reproducibility.

Formal definition. For a set of N independent simulation runs with random seeds $\{r_1, \ldots, r_N\}$, let $\{y_1, \ldots, y_N\}$ be the primary metric values. The validation gate requires:

$$\sigma_y < \tau \tag{6}$$

where τ is a pre-specified variance threshold. Additionally, all individual runs must satisfy bound constraints:

$$\forall j: \quad y_j \in [y_{\min}, y_{\max}] \tag{7}$$

Effect persistence is required across increasing sample sizes. An effect observed at $N = k$ must replicate at $N > k$ before acceptance.

Rationale. Stochastic simulations can produce apparent effects from sampling noise. A single run, or even three runs, may yield a "significant" result that vanishes at larger sample sizes. The variance gate ensures reproducibility; the escalation requirement catches false positives.

CHP implementation. Layer 6 (Sigma-Gated Verification). Default: 3 seeds for development, 30 seeds for convergence validation. Threshold $\tau = 0.15$ for standard deviation across seeds. No result is accepted on the basis of a single run.

SIMSIV instantiation. Four milestones $\times$ 30 seeds, 50 simulated years each (120 runs total). $\sigma = 0.030$, CV = 6.0%, all metrics within specification bounds. Section 4.8 documents a false positive caught by this component: an interaction effect of $+0.039$ at $n = 3$ seeds that vanished at $n = 10$ ($p = 0.954$).

3.5 Component 5: Persistent State and Termination Control

Objective. Prevent repeated errors and uncontrolled iteration.

Formal definition. The loop maintains three external state artefacts:

1. **State vector $\mathbf{s}^{(t)}$**: a compressed representation of the current loop state (turn, mode, milestone, metrics, flags), written every k turns and read at the start of every turn.

2. **Innovation log $\mathcal{L}$**: an append-only record of every build attempt, including decisions made, metrics achieved, and dead ends encountered.

3. **Dead-end register $\mathcal{D}$**: a set of failed approaches with reasons. The generation agent reads $\mathcal{D}$ before every turn and is prohibited from repeating any logged dead end.

The loop terminates when any of five conditions is met:

$$\text{Exit} = \text{ScienceComplete} \vee \text{Plateau}(N) \vee \text{Anomaly}(K) \vee \text{Misalignment} \vee \text{HumanStop} \tag{8}$$

where Plateau(N) fires after N consecutive turns with no metric improvement, and Anomaly(K) fires after K consecutive statistical gate failures.

Rationale. LLMs have no persistent memory across sessions and no intrinsic stopping criterion. Without external state, errors recur across sessions and loops run indefinitely. The dead-end register is particularly important: it converts discovered failures into permanent knowledge that prevents the model from repeating them.

CHP implementation. Layer 5 (Context Window Management) and Layer 9 (Self-Correcting Loop). Three files—`state_vector.md`, `innovation_log.md`, `dead_ends.md`—persist outside the LLM's context window and are read at the start of every generation turn.

SIMSIV instantiation. The SIMSIV repository contains `state_vector.md`, `V2_INNOVATION_LOG.md`, and `v2_dead_ends.md` documenting every decision, failure, and correction across the 11-turn autonomous build process.

4 Empirical Validation: The SIMSIV Experiment

4.1 Codebase Description

SIMSIV (https://github.com/kepiCHelaSHen/SIMSIV, DOI: 10.5281/zenodo.19065475) is a calibrated agent-based simulation of band-level human social evolution. Version 2: 7,663 lines,

35 heritable traits, 9 engine files, 187 tests. The simulation models gene-culture coevolution in pre-state societies, with cooperation, trade, raiding, institutional differentiation, and multi-level selection as emergent dynamics. Calibration is assessed against 9 anthropological benchmarks derived from the ethnographic literature (e.g., band sizes of 25–50, fission thresholds, cooperation ranges consistent with hunter-gatherer data). Each benchmark is scored pass/fail based on whether the simulated metric falls within the empirically documented range. A calibration score of 1.000 (9/9 benchmarks satisfied) indicates that all outputs fall within documented ethnographic bounds; uncalibrated runs with drifted coefficients typically score 0.44–0.67, failing on cooperation levels, group sizes, or both. SIMSIV is currently under review at JASSS (Journal of Artificial Societies and Social Simulation; submission 2026:81:1).

4.2 Three-Condition Experimental Design

Table 3: Three-condition experimental design.

Condition	Source Visible?	Protocol?	Models	n
A: Blind	No	No	GPT-4o, Grok-3	10/model/task
B: Source-informed	Yes (excerpt)	No	GPT-4o, Grok-3	10/model/task
C: Full protocol	Yes (full)	Yes (B/C/R)	Claude	1 build, 120 runs

Condition A represents a competent developer using an LLM: the prompt describes what to build (trait names, value ranges, scientific motivation) but does not include the source code containing the correct coefficients. This isolates the LLM's generative prior.

Condition B adds the relevant source code excerpt to the prompt (e.g., lines 287–292 of `resources.py` showing `empathy_capacity * 0.15`). This tests whether models use provided context when available.

Condition C is the full framework operating on the SIMSIV codebase: three adversarial roles (Builder generates, Critic validates against frozen specification, Reviewer checks code quality) with sigma-gated statistical validation. Condition C evaluates a single integrated build under the full protocol, with correctness assessed across 120 independent simulation runs (4 milestones $\times$ 30 seeds) rather than repeated generation trials. This reflects the framework's intended use case: a single disciplined build process producing validated output, not repeated sampling from an unconstrained generative distribution.

All Condition A and B trials used temperature 0.7 to ensure genuine variation across trials.

4.3 Condition A Results: 99% Drift Rate

Table 4: Drift rates under Condition A ($n = 10$ trials per model, temperature 0.7, no source code visible).

Coefficient	Truth	GPT-4o Drift	GPT-4o Mean	Grok-3 Drift	Grok-3 Mean
empathy	0.15	8/8 (100%)	0.362	10/10 (100%)	0.235
coop_norm	0.10	10/10 (100%)	0.235	10/10 (100%)	0.165
social_skill	0.10	10/10 (100%)	0.310	10/10 (100%)	0.300
cohesion_bonus	0.20	8/8 (100%)	0.462	10/10 (100%)	0.320
n_traits	4	9/10 (90%)	5.0	10/10 (100%)	7.4

Aggregate: 95 of 96 measurements drifted (**99.0%**). Some coefficients show fewer than 10 trials (e.g., empathy 8/8 for GPT-4o) because automated regex extraction failed to parse the coefficient from 2 responses; raw responses are archived for manual verification. Fisher's

exact test comparing Condition A (95/96 drifted) to Condition C (0/7 committed drift): $p = 4.0 \times 10^{-10}$.

Every drifted output was syntactically valid Python. Every drifted output would pass standard unit tests (e.g., "returns a float between 0 and 1"). Every drifted output would pass integration tests (the simulation runs correctly with wrong coefficients—it simply produces different, incorrect dynamics). Only direct comparison against the frozen specification detects the error.

4.4 Drift Is Systematic, Not Random

Table 5: Distribution of produced values for selected coefficients (Condition A).

Coefficient	Truth	Model	Min	Max	Mean	Correct	Character
empathy	0.15	GPT-4o	0.20	0.50	0.362	0/8	Variable, inflated
empathy	0.15	Grok-3	0.20	0.25	0.235	0/10	Concentrated, inflated
social_skill	0.10	GPT-4o	0.20	0.50	0.310	0/10	Variable, inflated
social_skill	0.10	Grok-3	0.30	0.30	0.300	0/10	**Locked** (zero var.)
n_traits	4	GPT-4o	4	6	5.0	1/10	Slight overcount
n_traits	4	Grok-3	7	8	7.4	0/10	Severe overcount

Grok-3 produced `social_skill_coeff = 0.30` on all 10 trials at temperature 0.7. The range was $[0.30, 0.30]$—zero variance. The correct value (0.10) was entirely outside its output distribution. This is not sampling noise; 0.30 is what the model's prior *believes* a social skill coefficient should be.

GPT-4o's drift was more variable but systematically inflated: no trial produced a value below the specification for any coefficient. The distribution is shifted upward, not scattered randomly around the true value.

Inter-model convergence. Models drifted to *different* wrong values for empathy (GPT-4o: 0.362, Grok-3: 0.235) but converged on *similar* wrong values for social_skill (0.310 vs 0.300) and cohesion (0.462 vs 0.320, overlapping ranges). This suggests partially shared training priors with model-specific concentration—consistent with models trained on overlapping but non-identical corpora.

A practical consequence: the more carefully calibrated a parameter is, the more vulnerable it is to drift. Standard values (0.50, 0.30) happen to be close to LLM priors. Unusual values (0.10, 0.15)—precisely those requiring careful empirical calibration—are the ones most reliably overwritten.

4.5 Condition B: Source Access Eliminates Coefficient Drift

Table 6: Condition A vs Condition B drift rates (reliable coefficients only).

Coefficient	Model	Condition A	Condition B	Change
empathy	GPT-4o	8/8 (100%)	0/10 (0%)	Eliminated
empathy	Grok-3	10/10 (100%)	0/10 (0%)	Eliminated
coop_norm	GPT-4o	10/10 (100%)	0/10 (0%)	Eliminated
coop_norm	Grok-3	10/10 (100%)	0/10 (0%)	Eliminated
social_skill	GPT-4o	10/10 (100%)	0/6 (0%)	Eliminated
social_skill	Grok-3	10/10 (100%)	1/9 (11%)	Nearly eliminated

When the source code excerpt containing the coefficient is visible in the prompt, both models produce correct values on ≥89% of trials. This confirms that **drift occurs because models generate from training priors when the specification is absent, not because they are incapable of reading specifications.** Drift is a context problem, not a capability problem.

However, Condition B does not prevent structural errors. The SIMSIV Critic caught three error types that source-code access alone does not address:

1. **Formula structure.** The Builder proposed a multiplicative CAC formula (producing degenerate zeros when density starts near zero) rather than the specification's additive decomposition.

2. **Scope creep.** The Builder proposed using all 35 heritable traits for selection analysis rather than the specification's 4 prosocial traits.

3. **Cross-file consistency.** The Builder used a conformity coefficient of 0.40 that was inconsistent with `institutions.py:237` (correct value: 0.30)—a reference not included in the source excerpt provided under Condition B.

These structural errors require *active enforcement* (the Critic cross-referencing multiple source files against the specification), not passive context inclusion.

4.6 Condition C: Full Protocol—Zero Committed Drift

Table 7: Critic friction events during SIMSIV-V2 construction (Condition C).

#	Milestone	Builder Proposed	Specification	Source	Error Type
1	M1	empathy = 0.20	0.15	`resources.py:289`	Coefficient
2	M1	conformity = 0.40	0.30	`institutions.py:237`	Coeff. (cross-file)
3	M1	CAC = multiplicative	additive	`resources.py`	Formula structure
4	M2	social_skill = 0.15	0.10	`clan_trade.py:330`	Coefficient
5	M3	cohesion = 0.25	0.20	`clan_raiding.py:610`	Coefficient
6	M4	35 traits	4 traits	`clan_selection.py:82-87`	Scope

All 6 proposals were blocked by the Critic and corrected before commit. **Zero of 7 committed coefficients drifted.** The Builder's proposed empathy coefficient (0.20) falls within Grok-3's Condition A range (0.20–0.25), confirming that the Builder exhibits the same drift as unstructured models. The framework does not prevent the proposal of drifted values—it intercepts them before they enter the codebase.

This is the key empirical result: the Builder, operating under the same generative prior as unstructured GPT-4o and Grok-3, produces the same drifted values. The framework's contribution is not preventing drift at the generation stage but catching it at the validation stage. The creative generation (code structure, naming, architecture) survives; only the specification-violating elements are filtered.

4.7 Multi-Seed Validation

The complete Condition C output was validated across 120 independent simulation runs (4 milestones × 30 seeds, 50 simulated years each).

All primary metrics satisfied $\sigma \ll 0.15$ (the framework's variance threshold). Zero regressions against 187 existing tests. Shapiro-Wilk normality confirmed for 4 of 5 metrics.

Table 8: Cross-seed convergence statistics (120 runs).

Milestone	Metric	$\bar{x}$	s	95% CI	CV	Normal
M1	cooperation	0.4994	0.0298	[0.488, 0.511]	6.0%	Yes
M1	CAC	0.6794	0.0458	[0.662, 0.697]	6.7%	Yes
M2	trade_capacity	1.0028	0.0152	[0.997, 1.009]	1.5%	No
M3	defence_capacity	0.6843	0.0403	[0.669, 0.699]	5.9%	Yes
M4	prosocial_composite	0.5008	0.0166	[0.495, 0.507]	3.3%	Yes

4.8 The False Positive Story—Proof of Component 4

This subsection documents a false positive caught by the framework's statistical validation component (Component 4). This event is the strongest evidence that the framework detects errors that would otherwise be published.

The initial finding. During SIMSIV-V2 Experiment 2 (institutional differentiation under inter-group conflict), the Builder reported a positive interaction effect of +0.039 between warfare intensity and institutional complexity on cooperation levels. The result was obtained from $n = 3$ random seeds. The interpretation was that the Bowles [2006] mechanism—war selects for in-group cooperation—appeared causally operative.

The sigma gate intervention. Component 4's variance gate flagged the result: $n = 3$ is underpowered for detecting interaction effects in stochastic simulations. The framework required replication at $n = 10$ before the result could be accepted. This escalation was automatic, following the pre-specified rule that effects must persist across increasing sample sizes [cf. Ioannidis, 2005].

The replication. At $n = 10$ seeds, the interaction effect was +0.0004 with $p = 0.954$. The original +0.039 effect was noise. The false positive was killed.

The dead end. The failed finding was logged in SIMSIV's `dead_ends.md`:

> *DEAD END: Interaction effect between warfare and institutions on cooperation. Appeared causal at n=3 (+0.039). Vanished at n=10 (p=0.954). Do NOT interpret n=3 interaction effects as causal.*

The real finding. The framework's dead-end register prevented the loop from revisiting the $n = 3$ false positive. The Builder scaled to $n = 20$ bands (increased statistical power) and found the real result: institutional complexity, not warfare, is the primary driver of sustained cooperation. The effect was $p < 0.0001$, Cohen's $d = -5.97$, direction State > Free (institutional societies showed higher cooperation) in 6/6 seeds.

Interpretation. The false positive was caught by the framework, not despite it. Without Component 4, the $n = 3$ result would have been accepted as evidence for the Bowles mechanism. The framework's variance gate forced the replication that killed the false positive, its dead-end register prevented the loop from returning to the failed interpretation, and its escalation protocol (increase n, try larger system size) led to the real finding. The complete error-correction sequence is documented in the SIMSIV git log and innovation log.

5 Discussion

5.1 Weakness as Strength

The framework's design rests on a non-obvious principle: the same property that causes specification drift—generation from training priors—also enables creative software generation. Suppressing the prior entirely (e.g., by feeding the model only the specification and demanding

verbatim reproduction) eliminates drift but also eliminates generative capability. The model becomes a copy-paste engine with no capacity for architectural innovation, naming conventions, or abstraction choices that exceed what the specification literally dictates.

The three-condition experimental design isolates this trade-off precisely. Condition A shows the prior unconstrained: rich creative generation, 99% drift. Condition B shows the prior partially constrained by context: correct coefficients, but structural errors persist because the model still generates architectural choices from its prior. Condition C shows the prior governed by adversarial roles: full creative generation *and* specification compliance, because the Critic selectively filters only the specification-violating elements while preserving the Builder's structural contributions.

Governing the prior through adversarial roles preserves both properties. The Builder generates freely, producing novel code structures, documentation, naming conventions, and architectural choices from its prior. The Critic inspects the result against the specification, correcting only the elements that violate the frozen ground truth. The result is code that is both creatively structured (Builder's contribution) and specification-compliant (Critic's contribution)—a combination that neither unstructured prompting nor specification-pasting achieves alone.

The evolutionary biology analogy is precise. Mutation is individually deleterious: most mutations are harmful, and most LLM-generated coefficients are wrong. Yet mutation is the substrate upon which natural selection operates: without it, there is no adaptation. Without the Builder's prior-driven generation, there are no novel proposals to evaluate. The Critic provides the selection pressure; the Reviewer provides the developmental constraint. The "phenotype"—committed code—contains only the variation that survived both filters.

5.2 Time Compression in Scientific Software Development

SIMSIV-V2 was constructed in 11 autonomous build turns. The v2 codebase comprises 7,663 lines across 9 engine files with 187 tests, satisfying all 9 anthropological calibration benchmarks (see Section 4.1). The human operator was asleep during the v2 build—the entire construction was fully automated, with the framework's persistent state management (Component 5) and termination control ensuring correct operation without human monitoring. When the operator woke up, the codebase was complete, all tests passed, and the innovation log documented every decision made overnight.

This is not an incidental detail. It provides direct evidence for the framework's sufficiency under autonomous operation. The Builder still proposed drifted coefficients during the overnight build (Table 7 documents these). The Critic still caught every one. The sigma gates still validated convergence. The dead-end register still prevented repeated mistakes. All of this happened without a human in the loop. The framework's value is not in augmenting human oversight—it is in *replacing* the need for continuous human oversight during scientific code generation.

A non-expert in evolutionary biology produced submission-quality simulation software in a weekend using this framework. The framework did not provide domain expertise; it provided the engineering controls (frozen specification, adversarial review, statistical gating) that ensured the LLM's creative generation remained aligned with the scientific specification. The domain expertise was front-loaded into the frozen specification; after that, the framework operated autonomously.

The implications for research acceleration are significant. The bottleneck in computational science is often implementation fidelity—translating a mathematical model into code that faithfully represents it. This translation traditionally requires both domain expertise (to understand the model) and software engineering skill (to implement it correctly). The framework decouples these: the domain expert writes the frozen specification, the LLM provides the creative implementation, and the validation loop ensures fidelity. The time from model specification to validated software collapses from weeks or months to hours.

5.3 Auditability as a Scientific Property

The framework produces a complete audit trail as a by-product of its operation:

- Every decision is logged in `V2_INNOVATION_LOG.md` with the turn number, mode, and metric deltas.
- Every coefficient is traced to a specific source file, line number, and literature citation.
- Every dead end is documented with the attempted approach, the reason for failure, and the "do not repeat" rule.
- The git log records every code change with the associated Critic review.

This audit trail is not a documentation burden imposed on the developer; it is a natural output of the framework's persistent state management (Component 5). The innovation log is append-only—it cannot be retroactively edited to hide failed approaches or ex-post rationalise decisions. The dead-end register provides a searchable record of every approach that was tried and abandoned, with reasons.

Anyone can clone the SIMSIV repository and independently verify: (a) which coefficients were specified, (b) which values the Builder proposed, (c) which the Critic caught, (d) what the final committed values were, (e) which dead ends were encountered and why, and (f) how the false positive was detected and corrected. This level of provenance—tracing every decision from specification to committed code through documented adversarial review—exceeds what most human-written scientific software provides. Reproducibility by strangers is achievable without any communication with the original developer.

The git log itself serves as a secondary audit mechanism. Each commit records the code changes and the associated build turn. The combination of git history and innovation log provides a complete, tamper-evident record of the development process—a property we term *computational provenance by construction.*

5.4 Limitations

Single-domain validation at scale. SIMSIV is the only production-scale validation (7,663 lines, 11 turns). The coefficients tested (0.10–0.20) are typical for agent-based models. Replication across other scientific domains—clinical trial criteria, financial risk models, climate simulations—would strengthen the generalisability claim.

Frozen specification requires domain expertise. Component 1 assumes the existence of a correct, complete specification. Writing this specification requires domain knowledge that the framework does not provide. The framework guarantees compliance with the provided specification; correctness therefore depends on the accuracy and completeness of the specification itself. If the specification is wrong, the framework will faithfully enforce the wrong values.

Multi-model council independence assumption. Component 2's adversarial separation is implemented within a single model using role prompts. Although Builder and Critic are instantiated within the same model, empirical results show systematic divergence between proposal and validation outputs: the Builder consistently proposed prior-consistent but specification-incorrect values (e.g., empathy = 0.20), while the Critic consistently identified and blocked these proposals by cross-referencing specific source-code lines. The separation is functional, not theoretical—the roles produce measurably different outputs. Nevertheless, the independence is assumed, not formally guaranteed. Cross-model councils (e.g., one model builds, a different model critiques) would provide stronger independence.

Verifiable oracle required for Component 4. Statistical validation (Component 4) requires a ground truth against which simulation outputs can be compared. Not all scientific domains have verifiable oracles. In domains where the "correct" output is unknown, Component 4 can verify reproducibility ($\sigma < \tau$) but not correctness.

5.5 Future Work

Cross-domain validation. Applying the framework to clinical trial simulation software (where coefficient drift could change dosage recommendations), financial risk models (where drifted parameters could alter capital requirements), and climate models (where calibrated feedback coefficients are critical to projections).

Automated frozen spec generation. Currently, the frozen specification is authored by a domain expert. Automated extraction of specifications from published papers, validated codebases, or regulatory documents would lower the barrier to adoption.

Quantifying council independence. Formal measurement of the statistical independence between Builder and Critic outputs, across both single-model role separation and cross-model configurations.

Formal complexity analysis. Characterising the termination conditions (Component 5) in terms of convergence guarantees and worst-case iteration bounds.

6 Conclusion

Large Language Models generate scientific code from training priors, not from user specifications. This is not a hypothetical risk: across 96 controlled measurements in this study, frontier models produced the correct coefficient exactly once. The drift is systematic (Grok-3 produced the same wrong value on 10/10 trials with zero variance), undetectable by standard software testing (every drifted output compiles, passes unit tests, and looks reasonable), and consequential (drifted coefficients change simulation dynamics without raising errors).

We present a five-component deterministic validation framework—Immutable Specification, Prior-Drift Detection, Adversarial Role Separation, Statistical Validation, and Persistent State with Termination Control—that treats prior-generation as a detectable signal rather than a defect to suppress. The framework's core mechanism is a context hack: injecting specification awareness into the LLM's active attention through adversarial role constraints. The Builder generates freely from its prior; the Critic validates against the frozen specification; the creative generation and the specification compliance occur in the same model, in the same session, through different roles.

Validated on SIMSIV—a 7,663-line scientific codebase under journal review—the framework produced zero committed specification drift across 7 checked parameters, validated over 120 simulation runs ($\sigma = 0.030$, CV $= 6.0\%$). The framework caught its own false positive ($n = 3$ interaction effect killed at $n = 10$, $p = 0.954$) before it could be published, then guided the loop to the real finding ($p < 0.0001$, Cohen's $d = -5.97$, 6/6 seeds). It enabled autonomous construction of production-quality scientific software by a non-domain-expert in a weekend. While validated on a single large-scale simulation, the framework's generality across domains with different specification structures remains to be tested. The framework guarantees compliance with the provided specification; correctness therefore depends on the accuracy and completeness of the specification itself. The framework is open-source (MIT License) and pip-installable.

Data Availability Statement

All code, data, frozen specifications, and experimental results are publicly available:

- **Framework:** `https://github.com/kepiCHelaSHen/context-hacking` (MIT License)
- **Validation codebase:** `https://github.com/kepiCHelaSHen/SIMSIV`
- **Zenodo DOI:** 10.5281/zenodo.19065475
- **Drift experiment data:** `experiments/drift_experiment/`

- **Innovation log:** `V2_INNOVATION_LOG.md`
- **Dead ends:** `v2_dead_ends.md`

References

Bai, Y., Kadavath, S., Kundu, S., Askell, A., Kernion, J., Jones, A., ... & Kaplan, J. (2022). Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*.

Bowles, S. (2006). Group competition, reproductive leveling, and the evolution of human altruism. *Science*, 314(5805), 1569–1572.

Boyd, R. & Richerson, P.J. (1985). *Culture and the Evolutionary Process*. University of Chicago Press.

de Waal, F.B.M. (2008). Putting the altruism back into altruism: The evolution of empathy. *Annual Review of Psychology*, 59, 279–300.

Ioannidis, J.P.A. (2005). Why most published research findings are false. *PLoS Medicine*, 2(8), e124.

Khattab, O., Singhvi, A., Maheshwari, P., Zhang, Z., Santhanam, K., Vardhamanan, S., ... & Zaharia, M. (2023). DSPy: Compiling declarative language model calls into self-improving pipelines. *arXiv preprint arXiv:2310.03714*.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., ... & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474.

Price, G.R. (1970). Selection and covariance. *Nature*, 227, 520–521.

Rice, J. (2026). SIMSIV: A calibrated agent-based framework for studying gene-culture coevolution in pre-state societies. *Under review, JASSS*, submission 2026:81:1.

Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K., & Yao, S. (2023). Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36.

Stodden, V., McNutt, M., Bailey, D.H., Deelman, E., Gil, Y., Hanson, B., ... & Taufer, M. (2016). Enhancing reproducibility for computational methods. *Science*, 354(6317), 1240–1241.

Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., ... & Zhou, D. (2022). Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35, 24824–24837.

Wiessner, P. (1982). Risk, reciprocity and social influences on !Kung San economics. In E. Leacock & R. Lee (Eds.), *Politics and History in Band Societies* (pp. 61–84). Cambridge University Press.

Zhou, A., Yan, K., Shlapentokh-Rothman, M., Wang, H., & Wang, Y.-X. (2023). Language agent tree search unifies reasoning, acting, and planning in language models. *arXiv preprint arXiv:2310.04406*.